

```
In [2]: import random, sys, time
sys.path.append('/opt/spark/python')
sys.path.append('/opt/spark/python/lib/py4j-0.10.9.7-src.zip')
```

```
In [3]: import pyspark
```

Programming 5 Lecture 4

Introduction to Spark

A MapReduce implementation -- and beyond



Spark is MapReduce on Steroids!

- Like MapReduce, it automatically distributes your work over a cluster (and also your data, if you want)
- But it has some advantages over MapReduce that make it the go-to platform now.



Spark's advantage: Iteration

- Unlike MapReduce implementations it can *iterate*

- This means your computations can *revisit* data
- This is crucial for e.g. machine learning

Spark's advantage: Graph based computation

- Spark calculates a computational graph based on the work and available nodes
- This means it can apply optimizations; unlike MapReduce it can *order* the computations
- This means it can also do *streaming* analytics (i.e. never storing the data at all)

Spark's advantage: DataFrames, No preferred datastore

- You can import data from any kind of source into Spark's dataframes
- E.g. import csv, xml, json
- Or build on huge existing stores in Hadoop Base (HBase), SQL databases or Distributed stores (Cassandra)

Partitions

Data is spread over the nodes in the cluster

- You can think of these as a bunch of rows from a dataframe
- These are called partitions
- They are also dynamically created according to the operations you carry out
- Note: these are *in memory*
- Also, they are usually set by the system; but if you specify them yourself, you can easily shoot yourself in the foot!

Narrow vs Wide operations

- A "narrow" operation transforms one partition into one other (can be done on a node)
- A "wide" operation transforms partitions making use of other partitions

Lazy Evaluation

- All transformations are queued until actually triggered by an *action*

- An action is something that "reports" on the transformed data
- for example, show() or count()
- Spark will analyze the entire pipeline to work on only the data needed at the end (by the action)

Three kinds of actions

- Actions to view data in the console
- Actions to collect data to native objects in the respective language
- Actions to write to output data sources

Work Plan

- Anytime you write a pipeline, you can ask Spark to "explain" (with the `.explain()` function) what it is that it will do to execute your pipeline.
- This can be very helpful with debugging

```
In [1]: from pyspark.sql import SparkSession
spark = SparkSession.builder.master("local[16]").appName("prog5_4").getOrCreate()
sc = spark.sparkContext
```

```
24/10/03 13:18:00 WARN Utils: Your hostname, nuc430.bin.bioinf.nl resolves to a loop
back address: 127.0.1.1; using 192.168.2.90 instead (on interface eno1)
24/10/03 13:18:00 WARN Utils: Set SPARK_LOCAL_IP if you need to bind to another addr
ess
Setting default log level to "WARN".
To adjust logging level use sc.setLogLevel(newLevel). For SparkR, use setLogLevel(ne
wLevel).
24/10/03 13:18:02 WARN NativeCodeLoader: Unable to load native-hadoop library for yo
ur platform... using builtin-java classes where applicable
```

See what's happening

<http://localhost:4040>

```
In [4]: nums = sc.parallelize([1,2,3,4])
nums
```

Out[4]: ParallelCollectionRDD[2] at readRDDFromFile at PythonRDD.scala:289

```
In [5]: squared = nums.map(lambda x: x*x).collect()
for num in squared:
    print('%i ' % (num))
```

[Stage 0:=====>

(5 + 11) / 16]

1
4
9
16

[Stage 0:=====>

(10 + 6) / 16]

In [7]: `import random`

`NUM_SAMPLES = 1000000000`

`def inside(p):`

`x, y = random.random(), random.random()`

`return x*x + y*y < 1`

`count = sc.parallelize(range(0, NUM_SAMPLES)).filter(inside).count()`

`print("Pi is roughly %f" % (4.0 * count / NUM_SAMPLES))`

[Stage 2:=====>

(15 + 1) / 16]

Pi is roughly 3.141585

In [5]: `from pyspark.sql import Row`
`from pyspark.sql import SQLContext`

`sqlContext = SQLContext(sc)`

`list_p = [('John',19),('Smith',29),('Adam',35),('Henry',50)]`

`rdd = sc.parallelize(list_p)`

`ppl = rdd.map(lambda x: Row(name=x[0], age=int(x[1])))`

`DF_ppl = sqlContext.createDataFrame(ppl)`

`DF_ppl.count()`

`DF_ppl.show()`

```
+-----+----+
| name|age|
+-----+----+
| John| 19|
|Smith| 29|
| Adam| 35|
|Henry| 50|
+-----+----+
```

A more sophisticated example

In [3]: `url = "https://bioinf.nl/~martijn/master/us_pop.csv"`

`from pyspark import SparkFiles`

`from pyspark.sql import SparkSession`

`from pyspark.sql import Row`

`from pyspark.sql import SQLContext`

`spark = SparkSession.builder.master("local[16]").appName("les5_example2").getOrCreate()`

`sc = spark.sparkContext`

```
sc.addFile(url)
sqlContext = SQLContext(sc)
SparkFiles.get("us_pop.csv")
```

24/06/20 08:38:16 WARN SparkContext: The path https://bioinf.nl/~martijn/master/us_pop.csv has been added already. Overwriting of added paths is not supported in the current version.

/homes/martijn/.venvs/default/lib/python3.11/site-packages/pyspark/sql/context.py:113: FutureWarning: Deprecated in 3.0.0. Use SparkSession.builder.getOrCreate() instead.

```
warnings.warn(
```

Out[3]: '/tmp/spark-32c9b6dc-fa85-404b-97ee-78e566be14ec/userFiles-76509eb9-ee1f-4d3d-bc42-6a40b6851cfc/us_pop.csv'

In [4]: `df = sqlContext.read.csv(SparkFiles.get('us_pop.csv'), header=True, inferSchema= True)`

In [8]: `df.printSchema()`

```
root
|-- x: integer (nullable = true)
|-- age: integer (nullable = true)
|-- workclass: string (nullable = true)
|-- fnlwgt: integer (nullable = true)
|-- education: string (nullable = true)
|-- educational-num: integer (nullable = true)
|-- marital-status: string (nullable = true)
|-- occupation: string (nullable = true)
|-- relationship: string (nullable = true)
|-- race: string (nullable = true)
|-- gender: string (nullable = true)
|-- capital-gain: integer (nullable = true)
|-- capital-loss: integer (nullable = true)
|-- hours-per-week: integer (nullable = true)
|-- native-country: string (nullable = true)
|-- income: string (nullable = true)
```

In [5]: `df.show(5)`

```

+---+---+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+-----+
| x|age|workclass|fnlwgt| education|educational-num| marital-status| occ
upation|relationship| race|gender|capital-gain|capital-loss|hours-per-week|native-co
untry|income|
+---+---+-----+-----+-----+-----+-----+-----+
-----+-----+
| 1| 25| Private|226802| 11th| 7| Never-married|Machine-op
-inspct| Own-child|Black| Male| 0| 0| 40| United-S
tates| <=50K|
| 2| 38| Private| 89814| HS-grad| 9|Married-civ-spouse| Farming-
fishing| Husband|White| Male| 0| 0| 50| United-S
tates| <=50K|
| 3| 28|Local-gov|336951| Assoc-acdm| 12|Married-civ-spouse| Protecti
ve-serv| Husband|White| Male| 0| 0| 40| United-S
tates| >50K|
| 4| 44| Private|160323|Some-college| 10|Married-civ-spouse|Machine-op
-inspct| Husband|Black| Male| 7688| 0| 40| United-S
tates| >50K|
| 5| 18| ?|103497|Some-college| 10| Never-married|
?| Own-child|White|Female| 0| 0| 30| United-States|
<=50K|
+---+---+-----+-----+-----+-----+-----+-----+
-----+-----+-----+-----+-----+-----+-----+
-----+-----+
only showing top 5 rows

```

```
In [6]: df.select('age', 'fnlwgt').show(5)
```

```

+---+-----+
|age|fnlwgt|
+---+-----+
| 25|226802|
| 38| 89814|
| 28|336951|
| 44|160323|
| 18|103497|
+---+-----+
only showing top 5 rows

```

```
In [8]: df.describe().show()
```

```

24/06/20 08:38:57 WARN SparkStringUtils: Truncated the string representation of a pl
an since it was too large. This behavior can be adjusted by setting 'spark.sql.debu
g.maxToStringFields'.
[Stage 4:=====> (1 + 1) / 2]

```

```

+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+
|summary|          x|          age|  workclass|          fnlwgt|  edu
cation|  educational-num|marital-status|  occupation|relationship|
race|gender|  capital-gain|  capital-loss|  hours-per-week|native-country|i
ncome|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+
|  count|          48842|          48842|          48842|          48842|
48842|          48842|          48842|          48842|          48842|          4
8842| 48842|          48842|          48842|          48842|          48842|
48842|
|  mean|          24421.5| 38.64358543876172|          NULL|189664.13459727284|
NULL|10.078088530363212|          NULL|          NULL|          NULL|          N
ULL|  NULL|1079.0676262233324|87.50231358257237|40.422382375824085|          NULL|
NULL|
|  stddev|14099.615260708357|13.710509934443525|          NULL|105604.02542315757|
NULL| 2.570972755592259|          NULL|          NULL|          NULL|          N
ULL|  NULL| 7452.019057655418| 403.004552124359|12.391444024252312|          NULL|
NULL|
|  min|          1|          17|          ?|          12285|
10th|          1|  Divorced|          ?|  Husband|Amer-Indian-Esk
imo|Female|          0|          0|          1|          ?| <
=50K|
|  max|          48842|          90|Without-pay|          1490400|Some-c
ollege|          16|  Widowed|Transport-moving|          Wife|
White|  Male|          99999|          4356|          99|  Yugoslavia|
>50K|
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+-----+-----+-----+-----+-----+
+-----+

```

```
In [12]: df.select('age', 'fnlwgt').show(5)
```

```

+---+-----+
|age|fnlwgt|
+---+-----+
| 25|226802|
| 38| 89814|
| 28|336951|
| 44|160323|
| 18|103497|
+---+-----+
only showing top 5 rows

```

```
In [13]: df.groupBy("education").count().sort("count", ascending=True).show()
```

```
+-----+-----+
| education|count|
+-----+-----+
|  Preschool|   83|
|    1st-4th|  247|
|    5th-6th|  509|
|  Doctorate|  594|
|      12th|  657|
|      9th|  756|
| Prof-school|  834|
|    7th-8th|  955|
|      10th| 1389|
|  Assoc-acdm| 1601|
|      11th| 1812|
|  Assoc-voc| 2061|
|    Masters| 2657|
|  Bachelors| 8025|
|Some-college|10878|
|    HS-grad|15784|
+-----+-----+
```

```
In [14]: df.describe('capital-gain').show()
```

```
+-----+-----+
|summary| capital-gain|
+-----+-----+
| count|          48842|
| mean|1079.0676262233324|
| stddev| 7452.019057655418|
| min|              0|
| max|          99999|
+-----+-----+
```

```
In [15]: df.crosstab('age', 'education').sort('age_education').explain()
```

```
== Physical Plan ==
*(1) Sort [age_education#1356 ASC NULLS FIRST], true, 0
+- Exchange rangepartitioning(age_education#1356 ASC NULLS FIRST, 200), ENSURE_REQUIREMENTS, [id=#180]
   +- LocalTableScan [age_education#1356, 10th#1407L, 11th#1408L, 12th#1409L, 1st-4th#1410L, 5th-6th#1411L, 7th-8th#1412L, 9th#1413L, Assoc-acdm#1414L, Assoc-voc#1415L, Bachelors#1416L, Doctorate#1417L, HS-grad#1418L, Masters#1419L, Preschool#1420L, Prof-school#1421L, Some-college#1422L]
```

```
In [16]: df.filter(df.age > 40).count()
```

```
Out[16]: 20211
```

```
In [17]: df.filter(df.education == "Preschool").count()
```

```
Out[17]: 83
```

```
In [18]: df.groupby('marital-status').agg({'capital-gain': 'mean'}).show()
```

```
+-----+-----+
| marital-status | avg(capital-gain) |
+-----+-----+
| Separated      | 581.8424836601307 |
| Never-married  | 384.382639449029 |
| Married-spouse-ab... | 629.0047770700637 |
| Divorced       | 793.6755615860094 |
| Widowed        | 603.6442687747035 |
| Married-AF-spouse | 2971.6216216216217 |
| Married-civ-spouse | 1739.7006121810625 |
+-----+-----+
```

```
In [19]: from pyspark.sql.functions import *
# 1 Select the column
age_square = df.select(col("age")**2)

# 2 Apply the transformation and add it to the DataFrame
df = df.withColumn("age_square", col("age")**2)
```

```
In [20]: df.printSchema()
```

```
root
|-- x: integer (nullable = true)
|-- age: integer (nullable = true)
|-- workclass: string (nullable = true)
|-- fnlwgt: integer (nullable = true)
|-- education: string (nullable = true)
|-- educational-num: integer (nullable = true)
|-- marital-status: string (nullable = true)
|-- occupation: string (nullable = true)
|-- relationship: string (nullable = true)
|-- race: string (nullable = true)
|-- gender: string (nullable = true)
|-- capital-gain: integer (nullable = true)
|-- capital-loss: integer (nullable = true)
|-- hours-per-week: integer (nullable = true)
|-- native-country: string (nullable = true)
|-- income: string (nullable = true)
|-- age_square: double (nullable = true)
```

```
In [21]: df.drop('race').columns
```

```
Out[21]: ['x',
          'age',
          'workclass',
          'fnlwgt',
          'education',
          'educational-num',
          'marital-status',
          'occupation',
          'relationship',
          'gender',
          'capital-gain',
          'capital-loss',
          'hours-per-week',
          'native-country',
          'income',
          'age_square']
```

```
In [102... sc.stop()]
```

References

- [Apache Spark Project](#)
- [Python Examples](#)
- [AMP camp](#)

Books

[Spark: The Definitive Guide](#)", O'Reilly Associates 2018, Bill Chambers & Matei Zaharia

```
In [ ]:
```

1. DataFrame Methods (No Import Needed)

- o What it is: A list of every method you can call on a DataFrame.
- o What it's for: When you're thinking, "How do I... do something to my DataFrame?"
- o Examples: `select()`, `filter()`, `groupBy()`, `orderBy()`, `withColumn()`, `describe()`, `show()`.

2. `pyspark.sql.functions`

- o What it is: A list of all the functions you can import and use on columns
- o What it's for: When you're thinking, "How do I... calculate or express something?"
- o Examples: `col()`, `count()`, `avg()`, `sum()`, `when()`, `concat()`, `lit()`.

The crucial takeaway is that the DataFrame methods orchestrate the process (Methods directly attached to DataFrame object e.g., `df.method()`)

while the SQL functions provide the specific building blocks for mathematical and logical expressions on the data within the columns.

Essential PySpark DataFrame Methods: These methods are attached directly to the DataFrame object (df.method()) and define the structure of your data transformation pipeline.

1. Selecting and Filtering Data		
Method	Purpose	Lecture Example
.select()	To select a subset of columns from the DataFrame.	df.select('age', 'fnlwgt')
.filter()	To select a subset of rows based on a conditional expression. (Also aliased as .where())	df.filter(df.age > 40)
2. Creating and Removing Columns		
Method	Purpose	Lecture Example
.withColumn()	To add a new column to the DataFrame or replace an existing one, usually based on a calculation.	df.withColumn("age_square", col("age")**2)
.drop()	To remove one or more columns from the DataFrame.	df.drop('race')
3. Grouping and Aggregation		
Method	Purpose	Lecture Example
.groupBy()	To group the DataFrame data by one or more columns, preparing it for aggregation.	df.groupBy("education")
.agg()	To apply one or more aggregation functions (like mean, count, sum) to the grouped data.	df.groupby(...).agg({'capital-gain': 'mean'})
4. Viewing and Execution (Actions)		
Method	Purpose	Lecture Example
.show()	The primary Action to display the contents of the DataFrame in the console.	df.show(5)
.count()	An Action that returns the number of rows in the DataFrame.	df.filter(...).count()
.printSchema()	Displays the column names and data types of the DataFrame (useful for debugging schema issues).	df.printSchema()

Essential PySpark SQL Functions: These functions are imported from pyspark.sql.functions and are used inside the methods listed above to build logical expressions, especially for column transformations and aggregations.

Function	Import & Purpose	Used Within Method
col()	Represents a DataFrame column when building an expression, allowing mathematical operations.	.select(), .withColumn(), .filter()
avg() / mean	Aggregation function to calculate the average of a column.	.agg(), .describe()
count()	Aggregation function to count the number of values in a column (distinct or non-distinct).	.agg(), .groupBy()
sum()	Aggregation function to calculate the sum of a column.	.agg()